

Analysis of memory mechanisms in LLMs (context window, KV-cache, retrieval-augmented memory)

1st Aayush Chhuka *Department of Computer Science and Engineering*
National College of Engineering
 Lalitapur, Nepal
 aayushchhuka20@gmail.com

2nd Aditya Brajacharya *Department of Computer Science and Engineering*
National College of Engineering
 Lalitapur, Nepal
 aadityabrjacharya123@gmail.com

3rd Sudip Dhungana *Department of Computer Science and Engineering*
National College of Engineering
 Lalitapur, Nepal
 sudipdhungana02@gmail.com

I. INTRODUCTION

Recently, Large Language Models (LLMs) have become a core foundation in AI-driven applications due to the sole reason of powerful understanding and generation of natural language. Unlike traditional AI systems, LLMs provide great flexibility and handle wide range of tasks with enhanced adaptability [1]. To manage active information, LLMs utilize a short-term memory technique known as the context window. Within this window, the model processes previous conversational turns, external documents, and its own prior outputs. To generate logical text, LLMs rely on an autoregressive process which generates tokens one by one based on all previous tokens to understand the broader context of the sequence [2].

A main challenge for LLM deployment is how models manage, access and retain information across interactions. Due to this autoregressive behaviour, the computing cost of a model increases as it recalculates the mathematical relation of previous tokens each and every time a new token is generated [2]. This leads to computational overhead within the context window. In some cases input context can be tens and hundreds of times longer than the output it generates [3].

Consequently, optimizing these memory mechanisms through techniques such as KV-caching and Retrieval-Augmented Memory has become critical for efficient LLMs deployment [1]. To overcome the limitation of recomputation of past token, the key-value (KV) cache stores the intermediate attention states (Keys and Values) of past tokens in memory during the processing phase [4]. LLMs struggle and produce hallucinations when answering beyond training data. To overcome this challenge, Retrieval-Augmented Generation (RAG) enhances performance by retrieving relevant documents from external knowledge bases such as database, text files or live internet using semantic similarity to match the meaning of questions [5].

Although these mechanisms have significantly improved

the capabilities of LLMs, each technique introduces various trade-offs in terms of memory, latency and computational cost. With rapid development in large context model, it is difficult to compare, understand the pros and cons of different memory mechanisms. This report provides comprehensive analysis of memory mechanisms on modern LLMs. It reviews fundamentals behind context windows, KV cache optimization, and retrieval-augmented memory approaches. This report compares these technique in terms of their computational cost and inference time, highlighting their advantages, limitations, and suitable application scenarios.

II. LITERATURE REVIEW

A. Memory in Large Language Models

Standard Large Language Models are stateless, which means they process each prompts and requests independently without retaining any information across multiple different interactions. This limits LLM in personalized interaction and consistency in long term conversation. To overcome these challenges, recent studies have introduced various memory mechanisms to preserve necessary information across different interactions while reducing hallucinations through external information retrieval [1], [5].

Existing studies have categorized memory into Short Term Memory (STM) and Long Term Memory (LTM). STM is used to handle the input within the immediate context window which can be the text tokens or embeddings currently available for prompt processing. Once the session ends, the model usually “loses” that context. Similarly LTM are implemented through external databases, vector stores, or graph structures that keep relevant data available and allow the model to “recall” information in future queries [1].

B. Context Window

The context window is the amount of text that an LLM can remember and process at one time. It works like the model’s

short-term memory, allowing it to use previous information while creating a response. User questions, instructions, and earlier conversations all take up space in the context window. When the window becomes full, older information is removed, and the model can no longer use it for generating new responses [1].

Modern LLMs can support longer context windows, but more text doesn't mean better results. Liu et al. [6] found the "Lost in the Middle" problem, where models can find information more easily when it is at the start or end of the input but may miss information in the middle. This shows that making the context window bigger alone is not enough, and better ways to manage memory are needed.

C. KV Cache Optimization

KV cache optimization is crucial for LLM deployment as it reduces latency by avoiding recomputing past tokens, lowering operational costs with less compute requirements, and enables deployment on resource-constrained devices by managing large memory and optimizing data transfers. KV Cache stores intermediate key which represents identity of token and value vectors which represents actual content of that token from previous tokens to avoid redundant computation during generation [4]. And when new token is generated, the key and value pair are appended to the cache. The KV cache size is expressed as:

$$KV_{pertoken} = 2 \times H \times L \times B \times D \quad (1)$$

In Equation (1) [4] H is the number of attention heads; D the dimension of each head; L the number of transformer layers; B the bytes per element. These parameters are fixed for a given model, the KV per token is constant for a model.

$$KV_{cachesize} = KV_{pertoken} \times ContextLength \quad (2)$$

Equation (2) shows that the total KV cache memory increases linearly with the context length. As modern LLMs support context windows containing tens or hundreds of thousands of tokens, the KV cache becomes a major bottleneck in terms of GPU memory consumption, memory bandwidth, and inference latency [2], [4].

1) *Grouped-Query Attention (GQA)*: Although KV caching reduces redundant computation, the cache size increases linearly as the context content grows which eventually leads to high memory consumption. To deal with this, Ainslie et al. [7] introduces Grouped-Query Attention (GQA), which groups multiple query heads to share same key and value as in Fig. 1. This result in reducing cache size while losing little output quality.

The result from the experiment shows that GQA inference speed is close to that of Multi-Query Attention (MQA) and maintains performance similar to that of standard Multi-Head Attention. Author also introduces an effective way to upgrade existing MHA models into GQA models using mean pooling by taking small amount of time and cost to adapt [7].

2) *KV Cache Quantization*: KV Cache Quantization is the optimization technique which is used to reduce memory requirement to store key and value tensors. Normally, these tensors are stored in high precision format usually FP16, instead of that these cached tensors are represented in lower precision format such as FP8, INT8, FP4. This helps in decreasing memory footprint and allows longer context length while maintaining model accuracy [2].

To reduce memory usage, multiple studies have introduced different KV cache quantization methods. FP8 KV cache is widely used because of its balance in memory savings and in model accuracy. A newer method, NVFP4 KV cache has been introduced by NVIDIA, which stores key and value tensors in 4-bit floating point format, reducing KV cache memory by half compare to that of FP8 allowing longer context lengths, supports more users, and lowers memory bandwidth during decoding. Cached values are dequantized for attention computation and quantized again before being stored [2].

Another method is MixQuant which uses mixed precision quantization by providing different precision levels to different cached tensors based on their importance. This reduce memory while maintaining better accuracy than using low level precision for all tensors.

TABLE I
COMPARISON OF KV CACHE QUANTIZATION TECHNIQUES.

Method	Precision	Main Benefit	Trade-off
FP16	16-bit	Highest accuracy	Large memory usage
FP8	8-bit	Lower memory	Slight accuracy loss
NVFP4	4-bit	Smaller cache	Quantization overhead
MixQuant	Mixed precision	Better accuracy-memory trade-off	Higher complexity

Table I summarizes recent KV cache quantization techniques based on the findings in [2], [3].

Although KV Cache Quantization reduces memory usage, it reduces generation quality for a longer context because of using low precision tensor format. As a result, current research focuses on finding the right balance between memory efficiency and model accuracy to enable the practical deployment of modern LLMs [2], [3].

3) *KV Cache Compression*: When context length increases, KV cache occupies large portion of GPU memory. While quantization reduces the precision of cached tensors, KV cache compression decreases the amount of information stored within the key value representations by removing redundancy. It reduces memory consumption while preserving all necessary information for accurate result [1].

Many studies have provided different ways to reduce the size of KV cache. One way is low-rank compression, which stores the key and value information in a smaller form while keeping most important informations. Another method is KV merging, where similar key value pairs are combined which reduce the number

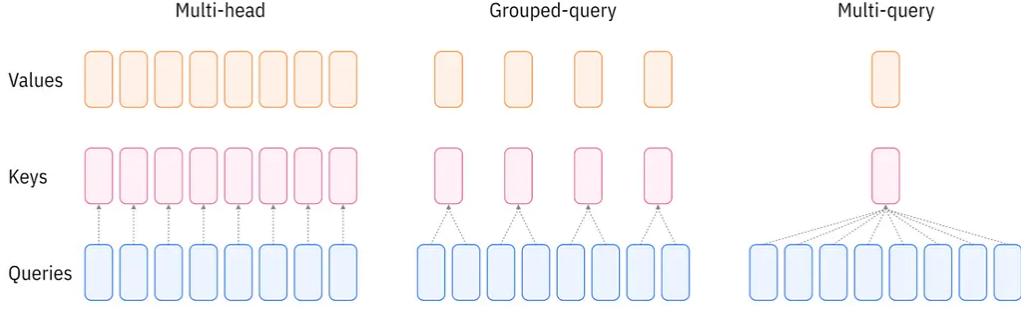


Fig. 1. An structural overview of the Grouped-Query Attention (GQA) mechanism compared alongside Multi-Head Attention (MHA) and Multi-Query Attention (MQA) [7].

of stored entries. Recent studies use multi-level compression by keeping important tokens in better quality while compressing less important or older tokens to save memory [1].

TABLE II
OVERVIEW OF KV CACHE COMPRESSION TECHNIQUES [1].

Technique	Principle	Advantage	Limitation
Low-Rank	Project KV tensors into lower-dimensional space	Lower memory usage	Reconstruction error
KV Merging	Merge similar KV pairs	Smaller cache size	Possible information loss
Multi-Level	Apply different compression levels	Better memory-quality balance	More complex implementation

Although KV cache compression uses less GPU memory and allow the model to handle longer input without using too much extra memory, compare to that of normal KV cache. If the data is compressed too much, some important information may be lost, which can reduce the quality of the output.

4) *Memory Management:* When input becomes longer, the KV cache also becomes larger, which increases memory usage. Memory management techniques helps to store, organize, and access the KV cache more efficiently during inference. Unlike quantization and compression, memory management does not reduce the KV cache size. Instead, it uses hardware memory more efficiently allowing longer inputs.

Offloading moves less frequently used parts of KV cache from GPU memory to other storage. When that data is needed, it is moved back to the GPU. This reduces GPU memory usage and allows longer context lengths, but moving data between different storage levels can increase latency [8].

PagedAttention uses similar method as virtual memory in operating systems by dividing the KV cache into small fixed-size pages. Instead of assigning a large memory space for each

request, it only uses memory pages. This reduces memory waste, and allows more requests to run together [9].

Shared Attention reduce memory usage by allowing different requests to share the same KV cache when they have similar prompts. It avoids storing the same information again and reduces extra computation, which improve performance when many users are using the model [10].

TABLE III
COMPARISON OF KV CACHE MEMORY MANAGEMENT TECHNIQUES.

Technique	Main Idea	Advantage	Limitation
Offloading	Move KV to CPU cache	Lower GPU memory usage	Transfer latency
PagedAttention	Page-based KV allocation	Better GPU utilization	More complex memory management
Shared Attention	Reuse common KV cache	Higher throughput	Requires shared contexts

D. Retrieval-Augmented Generation (RAG)

Large Language Models use knowledge from their training data, results they provided may be wrong or old. A larger context window doesn't fixed this problem as it still cannot get new information. Retrieval-Augmented Generation (RAG) first finds the documents and generate better answers using those documents [5].

A basic RAG system has three main parts: a retriever, a knowledge source, and a generator. The retriever finds useful information based on the user's questions, from sources like databases or documents. The retrieved information is added to the prompt, and the model uses it to generate more accurate answers [5].

RAG increases the accuracy of LLM by finding information from external sources. It reduces wrong answers without needing to train the model again. RAG acts like an external memory by storing information outside the model, so the model does not need to rely only on a larger context window [5].

The performance of RAG depends on the data source. If the information is falsy and misleading, generated output is affected. Not only that, RAG also takes more time and computation because it needs to search for information before generating a response. Recent studies focus on improving retrieval quality and reducing search time [5].

To overcome this, most common method is re-ranking. The retrieval of documentation is as usual. The documents are ranked based on the usefulness by ranking model before passing to LLM. Gao et al. [5] showed that re-ranking improves the quality of retrieved information, but it also requires extra computation. Another method is iterative method where model search information multiple times. This method is useful for complex questions that require information from multiple documents [5].

Self RAG, a newer method where model decide if it needs to extra information or not. Rather than searching for every question, it retrieves information when needed. Asai et al. [11] showed that this method reduces unnecessary retrieval and improves answer quality.

Graph-based retrieval is another way to improve RAG systems. Unlike normal retrieval methods, GraphRAG stores information as connected points and relationships in a knowledge graph. Instead of finding only separate text parts, it finds related information that helps the model understand connections between ideas. Edge et al. [12] showed that GraphRAG is useful for complex questions, but creating the knowledge graph needs extra work. Better RAG performance depends on not only finding similar text but also choosing useful information and organizing knowledge properly.

Compared with increasing the context window and improving the KV cache, RAG stores information outside the model instead of keeping everything in memory. It helps improve answer accuracy and reduce wrong information, but it also takes extra time because the model needs to search for information before answering. Therefore, RAG is useful for tasks where correct information is more important than fast response time.

REFERENCES

- [1] L. Shan, S. Luo, Z. Zhu, Y. Yuan, and Y. Wu, "Cognitive memory in large language models," 2025. [Online]. Available: <https://arxiv.org/abs/2504.02441>
- [2] E. Alvarez, W.-M. Chen, and H. Mao. (2025, Dec.) Optimizing inference for long context and large batch sizes with nvfp4 kv cache. NVIDIA. [Online]. Available: <https://developer.nvidia.com/blog/optimizing-inference-for-long-context-and-large-batch-sizes-with-nvfp4-kv-cache/>
- [3] H. Lu, Z. Chen, G. Fang, X. Ma, and X. Wang, "Mix-quant: Quantized prefilling, precise decoding for agentic llms," 2026. [Online]. Available: <https://arxiv.org/abs/2605.20315>
- [4] Y. Xu, N. K. Khaira, and T. Singh, "Kv cache optimization strategies for scalable and efficient llm inference," *arXiv preprint arXiv:2603.20397*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.20397>
- [5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, Q. Guo, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. [Online]. Available: <https://aclanthology.org/2024.tacl-1.9/>
- [7] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.
- [8] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, B. Chen, C. Barrett, J. E. Chen, J. Podolski *et al.*, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 31 094–31 116.
- [9] W. Kwon, Z. Li, S. Zhang, Y. Sheng, L. Zheng, C. H. Yu, T. Cao, B.-G. Jeon, K. Ramchandran, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [10] M. Fawaz, A. Awad, and W. El-Hajj, "Beyond kv caching: Shared attention for efficient llms," *arXiv preprint arXiv:2407.12866*, 2024.
- [11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://openreview.net/forum?id=hSyW5go0v8>
- [12] D. Edge, H. Trinh, Y. Cheng, J. Bradley, A. Chao, A. Mody, and J. Larson, "From local to global: A graphrag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.16130>